

현대적 저수준 그래픽스 API 기반 데이터 지향 멀티플랫폼 렌더링 프레임워크

dy_framework

AI 시대의 핵심 연산 자원, GPU.

그 가능성을 활용하려면 개발 기반도 달라져야 한다.

1. 작품의 개요 - 개발 배경과 필요성

인공지능의 학습과 추론, 과학 시뮬레이션, 실시간 그래픽스는 방대한 데이터를 병렬로 처리하는 GPU의 연산 능력을 활용한다. GPU의 역할이 확대되면서, 소프트웨어에는 연산을 병렬 작업으로 구성하고 필요한 데이터를 효율적으로 공급하는 설계가 요구되고 있다. 실제 GPU의 처리 성능은 연산량뿐 아니라 메모리 접근과 작업 분배에도 영향을 받는다. ^[1]

그러나 GPU를 활용하는 개발 방식은 플랫폼마다 다르다.

Vulkan·Direct3D 12·Metal은 현대 GPU를 세밀하게 제어하는 대표적인 저수준 그래픽스 API다. 이들은 자원 관리, 명령 기록, 동기화를 다루는 방식이 서로 달라, 여러 플랫폼을 대상으로 개발할 때 API별 구현과 실행 규칙을 함께 관리해야 한다.

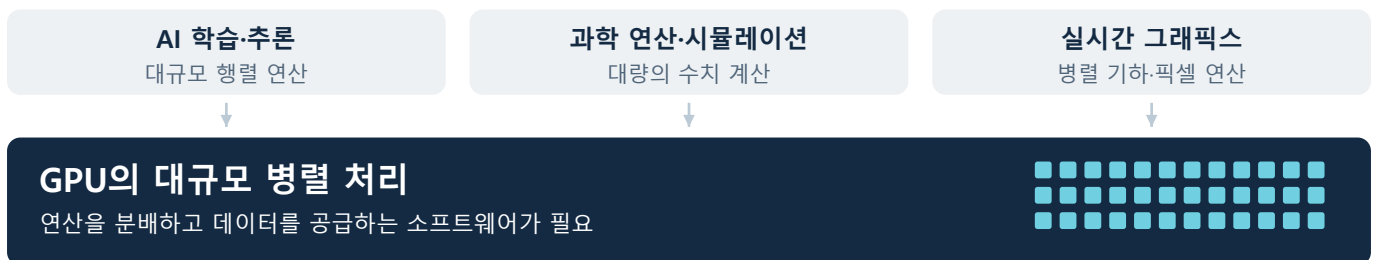
여러 운영체제에서 실행되는 것과, 각 환경의 GPU를 일관된 방식으로 저수준에서 제어하는 것은 서로 다른 설계 과제다. 플랫폼 간 이식성을 확보하면서도 GPU 자원과 실행에 대한 제어권을 유지하는 공통 기반이 필요한 이유다.

dy_framework는 이 과제를 다루는 C++ 렌더링 프레임워크다. Vulkan·Direct3D 12·Metal을 연결하는 공통 RHI를 중심으로, 데이터 지향 설계와 명시적 자원·명령 관리를 결합했다. 소프트웨어가 데이터를 준비하는 방식부터 GPU에 작업을 전달하는 규칙까지 설계 대상으로 삼았다.

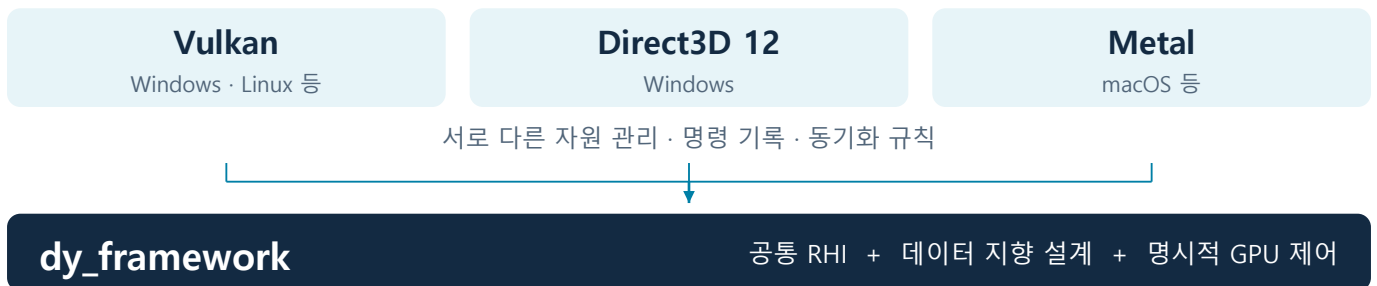
2. 작품의 구성 - 플랫폼별 GPU 제어를 연결하는 기반

GPU 활용은 확대되고, 제어 API는 분화되어 있다

기술적 배경



dy_framework가 다루는 영역: 현대적 저수준 그래픽스 API



상단: GPU 활용 분야의 예 / 하단: 본 작품의 API 통합 범위. 플랫폼은 대표 예시.

상위 렌더링 기능은 공통 인터페이스를 사용하고, 백엔드는 각 그래픽스 API의 규칙을 구현한다. 이 구조를 통해 플랫폼별 구현을 분리하면서, GPU 자원과 명령의 제어 방식을 상위 계층까지 연결한다.

3. 프로젝트 개발환경 및 개발자

C++17 · CMake · MSVC / GCC / Clang
HLSL · GLSL · Metal / Windows · Linux · macOS

한재승 - Core-RHI / 윤훈 - Vulkan

정현진 - Direct3D 12 / 정준혁 - Metal

4. 핵심기술 설명

GPU의 연산 능력을 활용하는 설계는 데이터·메모리·명령을 함께 다룬다.

① 현대적 저수준 API의 제어 모델을 공통 구조로

현대적 그래픽스 API에서는 개발자가 GPU 자원의 상태와 작업 간 동기화, 실행 중인 자원의 수명을 직접 다뤄야 한다. 예를 들어 Direct3D 12에서는 이전에 런타임이 암묵적으로 처리하던 CPU-GPU 동기화와 동적 자원 관리가 애플리케이션의 명시적인 책임이 되었다. [2]

따라서 여러 API를 연결하는 계층에서도 어떤 제어를 공통화하고, 어떤 실행 책임을 개발자에게 제공할 것인가를 설계해야 한다. 추상화 과정에서 발생하는 데이터 복사, 상태 추적, 메모리 할당 비용 역시 고려 대상이다. 이는 SIGGRAPH 2023의 HypeHype Mobile Rendering Architecture에서도 다루는 현대 렌더링 기반 설계의 핵심 문제다. [3]

dy_framework의 RHI는 GPU 자원, 그래픽스·컴퓨터 명령, 자원 상태 전이, 제출과 완료 확인을 공통 인터페이스로 표현한다. 플랫폼별 구현을 분리하면서도 명령과 자원의 제어 지점을 공개하여, 상위 시스템이 자체 실행 정책을 구성하도록 설계했다.

② 하드웨어의 데이터 접근을 고려한 데이터 지향 설계

대량의 데이터를 처리할 때는 계산식뿐 아니라 필요한 데이터가 메모리 어디에, 어떤 순서로 놓이는가도 중요하다. 데이터 지향 설계(DOD) [4]는 실제 입력 데이터와 수행할 연산, 하드웨어의 특성을 기준으로 프로그램 구조를 정하는 접근이다.

dy_framework는 내부 장면 데이터를 변환·재질 등 용도별 배열로 나누어, 필요한 필드를 연속적으로 접근하도록 구성했다. 수학 연산에는 SSE-NEON 기반 SIMD를 적용한다. GPU에 작업을 공급하는 CPU 측 데이터 처리까지 함께 고려한 설계다.

같은 데이터, 다른 메모리 접근

변환 데이터 T만 사용하는 연산의 배치 예시

객체별 배치 · AoS

T1	M1	L1	X1
T2	M2	L2	X2
T3	M3	L3	X3

필요한 필드가 다른 데이터 사이에 놓임

용도별 배치 · SoA

T1	T2	T3
M1	M2	M3
L1	L2	L3
X1	X2	X3

같은 연산에 필요한 필드를 연속 배치

T 변환 · M 재질 · L 조명 · X 기타 | CPU 데이터 준비를 설명하는 개념도, 성능 측정값 아님

③ 병렬 작업 준비와 비동기 실행을 고려한 자원 관리

CPU의 명령 준비와 GPU의 실행은 서로 다른 시점에 진행된다. 이를 연결하려면 작업 간 의존성, 명령의 제출 순서, 자원을 다시 사용할 수 있는 시점을 함께 관리해야 한다.

dy_framework는 독립적인 명령 목록의 병렬 기록, 제출 시 자원 상태 검증, GPU 완료에 따른 명령 목록 재사용을 구분하여 구성했다. 렌더 그래프는 이 구조 안에서 패스 간 의존성과 필요한 배리어를 계산한다.

이러한 실행 기반 위에 렌더러와 장면 기능을 연결하여, 플랫폼별 GPU 제어와 하드웨어를 고려한 데이터 처리를 하나의 프레임워크 안에서 다룬다.

[1] NVIDIA GPU 구조·성능 자료

[2] Microsoft의 API 설계 변화 설명

[3] SIGGRAPH 2023 · HypeHype Mobile Rendering Architecture

[4] CppCon · Mike Acton, Data-Oriented Design and C++